



# Seminário — Padrões de Projeto com Exemplo Guiado

**Grupo: A**

**Integrantes do grupo:**

Andreza Helen Batista Da Silva - 259085

Beatriz Cristina de Oliveira Jatobá - 240421

Dandara Melissa Alves - 274081

Lília Cordeiro França - 197102

## 1. NOME E CATEGORIA DO PADRÃO ESCOLHIDO

- **Nome:** Memento
- **Categoria:** Padrão Comportamental (Behavioral Pattern).

## 2. LINKS

**Link do Repositório:**

<https://github.com/licoor/seminario-si405-padrao-memento>

**Link do Vídeo no Youtube:**

<https://youtu.be/0YJgCYNr3Z0?feature=shared>

## 3. PROBLEMA DE PROJETO ABORDADO

Em diversas aplicações, como editores de texto, jogos e sistemas de formulários, é comum existir a funcionalidade de desfazer ações (Ctrl+Z). Uma implementação simples costuma armazenar diretamente os atributos internos do objeto responsável pelos dados da aplicação.

Essa abordagem apresenta problemas de projeto, pois outras classes passam a acessar e modificar informações internas do objeto por meio de getters e setters públicos. Como consequência, ocorre quebra do encapsulamento, aumento do acoplamento entre as classes e maior dificuldade de manutenção e evolução do sistema.



#### 4. JUSTIFICATIVA PARA O USO DO PADRÃO

O padrão Memento resolve esse problema permitindo que um objeto salve seu próprio estado interno sem expor seus detalhes de implementação para outras classes.

No exemplo desenvolvido, o Editor de Texto é responsável por criar um objeto Memento contendo seu estado atual. O Histórico apenas armazena esses objetos e os devolve quando o usuário solicita a operação de desfazer, sem conhecer o conteúdo armazenado.

Dessa forma, o encapsulamento é preservado e cada classe passa a possuir uma responsabilidade bem definida.

#### 5. EXPLICAÇÃO RESUMIDA DA SOLUÇÃO

O padrão Memento possui três participantes principais:

- **Originator (EditorTextoSimples):** objeto responsável por criar e restaurar seus próprios estados.
- **Memento:** objeto imutável que armazena uma cópia do estado do Originator.
- **Caretaker (Historico):** responsável apenas por armazenar e recuperar os Mementos, sem acessar seus dados internos.

Na implementação refatorada, o Editor cria um Memento sempre que um novo estado deve ser salvo. Quando o usuário solicita um desfazer, o Histórico retorna o último Memento armazenado e o próprio Editor restaura seu estado utilizando esse objeto.

Essa abordagem reduz o acoplamento entre as classes, preserva o encapsulamento, melhora a organização do código e facilita futuras extensões da aplicação.

#### 6. FONTES CONSULTADAS

NUZZI, Jones Roberto. Design Patterns — Parte 20 — Memento. Medium, 21 jan. 2020.

Disponível em:

<https://medium.com/@jonesroberto/design-patterns-parte-20-memento-6b30ad75b12f>.

REFACTORING.GURU. **Memento**. [S. l.]: Refactoring.Guru, [s.d.]. Disponível em:

<https://refactoring.guru/pt-br/design-patterns/memento>. Acesso em: 6 jul. 2026.